ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТ №1
ТАЙЛАН

Үйлдлийн систем /F.CSM302/

Сэдэв:	Sudoku thread validator Судалгаа, хэрэгжилт, туршилтын тайлан
Сурах бичиг:	Operating System Concepts, 4-р бүлэг
Хуваарилалтын код:	IV-2-5
Хичээл заасан багш:	Г.Ганбат /F.CS10/
Гүйцэтгэсэн баг:	Э.Сумъяасүрэн /B200900040/ Т.Мөнгөнтүлхүүр /B241910041/ Э.Тэмүүлэн /B231910037/ А.Марал-Эрдэнэ /B241940106/ Б.Ундрах /B221910803/ Ө.Зоригтбаатар /B242270078/

Улаанбаатар хот, 2026 он

Sudoku Solution Validator (Pthreads) — Manual

Хэрэглэгчийн ба operational заавар · C11 + POSIX Pthreads · Python HTTP bridge · Browser dashboard

Хэсэг 1. Хэрэглэгчийн заавар

1.1 Програмын зорилго

Энэ програм нь 9×9 судокугийн шийдэл зөв эсэхийг **олон worker thread-ээр зэрэгцээ шалгадаг**. Шалгалтын гол хөдөлгүүр нь C11 + Pthreads-ээр бичигдсэн `sudoku_validator` юм. Судоку зөв байх нөхцөл: 9 мөр, 9 багана, 9 ширхэг 3×3 дэд тор бүрд 1–9 цифр тус бүр яг нэг удаа орсон байх.

Програм гурван хэлбэртэй:

- **CLI** (`sudoku_validator`): командын мөрөөс шууд шалгана.
- **Dashboard** (`dashboard.html`): хөтөч дээр торыг засаж, бодит C хөдөлгүүрээр шалгуулна.
- **Сургалтын симуляц** (`ui/sudoku_threads.html`): thread-ийн хуваарилалтыг дүрслэн үзүүлнэ (бодит хэмжилт биш).

1.2 Дэмжих орчин

Зүйл	Шаардлага
Үйлдлийн систем	Windows 10/11, macOS (Apple Silicon бэлэн APP; Intel дээр шалгаагүй), Linux
Бэлэн EXE / APP ашиглах бол	Python, GCC суулгах шаардлагагүй
Эх кодоос ажиллуулах бол	Python 3, C compiler (GCC эсвэл Clang), make
Windows compiler	MSYS2 UCRT64 GCC (PATH-д нэмсэн байх)
Хөтөч	Chrome, Edge, Firefox-ийн шинэ хувилбар
Порт	`127.0.0.1:8765` (зөвхөн локал, интернет шаардахгүй)

1.3 Суулгах ба нээх алхам

A. Windows — бэлэн EXE (энгийн хэрэглэгчдэд санал болгоно)

1. v1.2.1 Release-ээс SudokuValidation_Windows_Ready.zip татна.
2. ZIP-ийг **Extract All** хийж бүрэн задална (ZIP дотроос шууд ажиллуулахгүй).
3. `SudokuValidation.exe`-г ажиллуулна. Хөтөч автоматаар нээгдэнэ.
4. SmartScreen анхааруулга гарч болно. Зөвхөн итгэлтэй эх сурвалжаас татсан файлыг ажиллуулна.

B. macOS — бэлэн APP (Apple Silicon)

1. GitHub Actions-ийн `Build macOS Sudoku app` workflow-ийн амжилттай run-ээс `SudokuValidation_Mac_Ready` artifact татна.
2. ZIP-үүдийг задлаад `SudokuValidation.app` нээнэ.
3. Apple гарын үсэг/notarization байхгүй тул анх удаа **System Settings → Privacy & Security → Open Anyway** дарах шаардлагатай байж болно.

C. Эх кодоос (Windows, macOS, Linux)

OS	Launcher	Шаардлага
Windows	`START_WINDOWS.bat`	Python + GCC
macOS	`START_MAC.command`	Python 3 + Clang
Linux	`START_LINUX.sh`	Python 3 + GCC/Clang

Launcher нь (1) хөдөлгүүрийг шаардлагатай бол хөрвүүлнэ, (2) серверийг асаана, (3) хөтчийг нээнэ. Ажиллах хугацаанд launcher-ийн цонхыг хаахгүй байна.

1.4 Оролт ба тохиргоо

CLI сонголтууд

Сонголт	Утга
`-f ФАЙЛ`	Файлаас унших (`-` = stdin). 81 цифр; `.` эсвэл `0` = хоосон; `#` = тайлбар; зай, `|`, `-`, `+`, `,` үл тооцно; эхэнд UTF-8 BOM байж болно
`-g SEED`	SEED-ээр санамсаргүй **зөв** судоку үүсгэнэ
`-e K`	K нүдийг санамсаргүй өөр цифрээр сольж алдаатай болгоно (0–81)
`-s R,C,V`	[R,C] нүдийг V болгоно (давтаж болно)
`-t MODE`	Thread-ийн горим: 11 (анхдагч), 27, 1
`-b REPS`	Горим бүрийн хурдыг REPS удаа давтаж харьцуулна
`-v` / `-q`	Дэлгэрэнгүй / зөвхөн дүн

`-f` ба `-g`-г хамт, мөн `-f`-г хоёр удаа өгч болохгүй.

Гаралтын код: `0` = судоку зөв · `1` = судоку буруу · `2` = оролт эсвэл системийн алдаа.

Оролтын файлын формат

Нэг мөрт (81 тэмдэгт) эсвэл 9 мөр×9 тэмдэгт хэлбэртэй байж болно. Жишээ нь:

```
534678912
672195348
198342567
859761423
426853791
713924856
961537284
287419635
345286179
```

Dashboard

- Торын нүд бүрийг гараар засна (0 = хоосон).
- **Горим** (1, 11, 27) сонгоно.
- «Шалгах» дарахад хөтөч `POST /api/validate` хүсэлтийг локал серверт илгээж, C хөдөлгүүрийн бодит үр дүнг авна.
- Үр дүнд: ерөнхий дүн (зөв/буруу), worker бүрийн төлөв, алдаатай нүднүүд (тодруулна), C thread-ийн хэмжсэн хугацаа (мкс).
- **Алхмаар үзүүлэх** хэсэг нь C-ээс буцсан үр дүнг тайлбарлан дахин тоглуулдаг; энэ нь OS thread scheduler-ийн live trace биш.

1.5 Ажиллуулах, зогсоох

Үйлдэл	Яаж
Ажиллуулах (эх код)	`START_WINDOWS.bat` / `START_MAC.command` / `START_LINUX.sh` эсвэл `python3 launch.py`
Гараар	`make` → `python3 server.py` → хөтчөөр `http://127.0.0.1:8765/dashboard.html` нээнэ
Зогсоох	Launcher-ийн терминалд **Ctrl+C**; бэлэн EXE/APP-д цонхоо хаана
Анхаарах	Хөтчийн таб хаахад сервер үргэлжлэн ажиллана

1.6 Thread-ийн стратеги

`-t`	Worker	Хуваарилалт
11	11	1 «бүх мөр», 1 «бүх багана», 9 дэд тор (сурах бичгийн санал)
27	27	Мөр бүрд 1, багана бүрд 1, дэд тор бүрд 1
1	1	27 бүсийг нэг worker шалгана (харьцуулах суурь)

Үр дүн буцаах. Глобал `int valid[27]` массив. i-р worker зөвхөн `valid[i]`-д бичнэ, тиймээс mutex хэрэггүй. `pthread_join()` бичилт parent-д харагдахыг баталгаажуулна. Массив 0-ээр эхэлдэг тул ажиллаж чадаагүй worker автоматаар «буруу» гэж тооцогдоно.

`valid[]`**-ийн байрлал (27 горим):** эхний 9 = мөр, дараагийн 9 = багана, сүүлийн 9 = дэд тор.

Шалгах арга. Бүс бүрт 10 битийн mask ашиглана: `v` цифр харагдвал `1<<v` бит асна. Бит аль хэдийн асаалттай байвал давхардал. Эцэст нь `mask == 0x3FE` бол 1–9 бүгд байна.

1.7 Жишээнүүд

Жишээ 1. Анхдагч жишээ (Figure 4.26)

```
./sudoku_validator
```

Зөв судоку, 11 thread. Гаралтын код 0.

Жишээ 2. Нэг нүд буруу

```
./sudoku_validator -f tests/invalid_one_cell.txt -t 27
```

Гаралтын код 1. `valid[]`-д тухайн мөр, багана, дэд тор гурав `0` болно.

Жишээ 3. Санамсаргүй судоку, 2 алдаатай

```
./sudoku_validator -g 42 -e 2 -v
```

Seed 42-оор зөв судоку үүсгээд 2 нүдийг өөрчилнө; `-v` thread бүрийн мөрийг хэвлэнэ.

Жишээ 4. Нэг нүдийг гараар солих, stdin

```
./sudoku_validator -s 5,5,0
echo "534678912..." | ./sudoku_validator -f -
```

Жишээ 5. Хурдны харьцуулалт

```
./sudoku_validator -b 2000
```

1.8 Түгээмэл алдаа ба шийдэл

Шинж тэмдэг	Шалтгаан	Шийдэл
`GCC not found` / `C compiler not found`	Compiler суугаагүй эсвэл PATH-д алга	Windows: MSYS2 UCRT64 GCC суулгаж PATH-д нэмнэ. macOS: Xcode Command Line Tools. Linux: `sudo apt install build-essential python3`
`Port 8765 is already in use`	Хуучин сервер ажиллаж байна	Хуучин цонхыг хааж дахин ажиллуулна
`C engine not built`	Binary үүсээгүй	`make` (macOS/Linux) эсвэл `build_windows.bat` ажиллуулна
Гаралтын код 2	Оролт буруу (80 нүд, үсэг, файл олдохгүй, буруу `-s`)	Оролтын формат, сонголтын утгыг шалгана
API 503 Сервер ачаалалтай	Зэрэг 4-өөс олон шалгалт	Хэсэг хүлээгээд дахин оролдоно
Хөтөч UI хуучин харагдана	Cache	**Ctrl+Shift+R**
Mac дээр апп нээгдэхгүй	Гарын үсэггүй апп	Privacy & Security → **Open Anyway**

Хэсэг 2. Operational заавар

2.1 Repository бүтэц

Файл	Үүрэг
`sudoku_validator.c`	Үндсэн C11 + Pthreads хөдөлгүүр
`Makefile`	`make`, `make tsan`, `make asan`, `make test`
`server.py`	Локал HTTP bridge: C binary-г ажиллуулж үр дүнг JSON-оор буцаана
`dashboard.html`	Үндсэн интерфэйс (хөтөч)
`ui/sudoku_threads.html`	Сургалтын симуляц
`launch.py`	Нэг товшилтын launcher: хөрвүүлэх → сервер → хөтөч
`launch_standalone.py`	Windows EXE-ийн entry point (PyInstaller `--onedir`)
`mac_app_launcher.py`	macOS APP-ийн entry point (PyInstaller)
`START_WINDOWS.bat`, `START_MAC.command`, `START_LINUX.sh`	OS тус бүрийн launcher
`run_windows.bat`, `run_linux.sh`	Хөрвүүлээд серверийг шууд асаах хялбар скрипт
`build_windows.bat`	Windows дээр GCC-ээр хөрвүүлнэ
`run_tests.sh`	Автомат тест
`tests/`	Тестийн оролтын файлууд ба Python unit тест
README.md docs/USER_GUIDE.md	Эхлүүлэх ба дэлгэрэнгүй заавар.

2.2 Build ба Run команд

	Windows	Linux / macOS
Build	`build_windows.bat`	`make`
Run (дашборд)	`run_windows.bat` эсвэл `START_WINDOWS.bat`	`./run_linux.sh` эсвэл `python3 server.py`
Run (CLI)	`sudoku_validator.exe -f tests\valid_fig426.txt`	`./sudoku_validator -f tests/valid_fig426.txt`
Хөрвүүлэлтийн flag	`gcc -std=c11 -Wall -Wextra -O2 -pthread`	ижил

Санамж: `launch.py` нь binary байхгүй эсвэл `.c` файл binary-аас шинэ бол автоматаар дахин хөрвүүлнэ.

2.3 Сервер ба API

Сервер 127.0.0.1 дээр ажиллана. Анхдагч порт 8765. --port-оор солино.

Endpoint	Үүрэг
`GET /`	`/dashboard.html` руу чиглүүлнэ
`GET /dashboard.html`, `/ui/sudoku_threads.html`	Зөвхөн эдгээр хоёр файлыг нийтэд нээлттэй өгнө (directory listing байхгүй)
`POST /api/validate`	Судоку шалгана

Хүсэлт: `Content-Type: application/json`

```
{ "board": [81 ширхэг 0–9 бүхэл тоо], "mode": 1 }
```

`mode` нь 1, 11, эсвэл 27 байна.

Харин: valid, mode, workers (label, ok), badCells (0–80), issues, microseconds, engine, note.

Хамгаалалтын хязгээр:

Хязгаар	Утга
Host header	127.0.0.1 эсвэл localhost, ажиллаж буй порттой (бусад нь 403)
Origin	Хоосон эсвэл дээрх хоёр (бусад нь 403)
Хүсэлтийн хэмжээ	1–8192 byte
Зэрэг ажиллах шалгалт	Дээд тал нь 4 (илүү бол 503)
C процессын timeout	8 секунд

Алдааны код: `400` оролт буруу · `403` host/origin буруу · `404` олдсонгүй · `415` JSON биш · `503` C engine байхгүй, timeout эсвэл хөдөлгүүрийн алдаа, эсвэл хэт олон хүсэлт · `500` серверийн алдаа.

C worker-уудын үр дүнг -j горим JSON-оор гаргана. badCells нь зөрчилтэй нүдний индекс, issues нь бүс бүрийн алдаа. Хоосон (0) нүд алдаанд тооцогдоно.

2.4 Тест

```
make test                              # хөрвүүлж + автомат тест (sanitizer-тэй)
bash run_tests.sh                      # тестийг шууд ажиллуулах
python3 -m unittest discover -s tests -v
STRESS=1000 bash run_tests.sh          # стрессийн тоог нэмэх (анхдагч 300)
REQUIRE_SANITIZERS=1 bash run_tests.sh # sanitizer алгасвал бүтэлгүйтэнэ
```

`run_tests.sh` дараах 4 хэсгээс бүрдэнэ:

1. **Файлын тест.** Горим бүрд (1, 11, 27) хүлээгдэж буй гаралтын код (0 зөв / 1 буруу / 2 алдаа): зөв файл, нэг нүд буруу, зөвхөн дэд тор буруу, зөвхөн багана буруу, бүрэн бус, бүгд 5, 80 нүд, үсэг, файл байхгүй, stdin, BOM, мөн буруу аргументууд (`-s`, `-e`, `-g`, `-t`, `-b`).
2. **Бүсийн тусгаарлалт.** 27 горимд `valid[]` массив яг аль бүс буруу болохыг зөв заах эсэх.
3. **Стресс.** `-g SEED` зөв судоку (код 0) ба `-g SEED -e K` алдаатай судоку (код 1), 3 горим.
4. **Sanitizer.** ThreadSanitizer (data race), AddressSanitizer + UBSan (санах ой). Binary байхгүй бол хэсгийг алгасна.

2.5 Хүлээгдсэн үр дүн

Оролт	Хүлээгдсэн
`tests/valid_fig426.txt`, `valid_compact.txt`, `valid_bom.txt`, анхдагч оролт	Код 0 (бүх 3 горимд)
`invalid_one_cell`, `invalid_subgrids_only`, `invalid_columns_only`, `invalid_incomplete`, `invalid_all_same`	Код 1
`bad_short` (80 нүд), `bad_char`, `bad_shape`, файл байхгүй, хавтас	Код 2
`-t 27` ба `invalid_one_cell`	`valid[]`-д мөр1, багана1, тор1 нь 0
`-t 27` ба `invalid_subgrids_only`	Зөвхөн сүүлийн 9 (дэд тор) 0

Хурдны хэмжилт (`-b 2000`, 2 vCPU Linux, gcc 13.3 -O2)

Горим	Thread	Дундаж (мкс)	1 thread-тэй харьцуулахад
`-t 1`	1	47.6	1.00×
`-t 11`	11	291.7	6.12× удаан
`-t 27`	27	672.4	14.12× удаан

Нэг судоку шалгах ажил хэдхэн микросекунд, харин thread үүсгэх, join хийх зардал (~15–25 мкс/thread) түүнээс их тул **энэ хэмжээний ажилд олон thread хурдсахгүй, харин удаашруулдаг**. Зэрэгцээ байдал нь олон мянган судокуг багцаар шалгах мэт том ажилд ашигтай.

2.6 Known issues

• 8765 порт завгүй бол эх кодоос python3 server.py --port 8766 гэж асаагаад http://127.0.0.1:8766/dashboard.html хаягийг нээнэ.

- macOS APP нь Apple Developer ID-аар гарын үсэггүй; Intel Mac дээр шалгаагүй.
- Windows EXE-д SmartScreen анхааруулга гарч болно.
- Бүх шалгалт серверийн талд C процесс ажиллуулдаг тул C binary заавал build хийгдсэн байх ёстой.

• Linux/WSL2 дээр TSan unexpected memory mapping гэж зогсвол, орчин зөвшөөрсөн үед setarch "$(uname -m)" -R make test гэж дахин ажиллуулна. TSan-ийн орчны алдааг амжилттай шалгалтад тооцохгүй.

• Эхлүүлэх заавар README.md, дэлгэрэнгүй заавар docs/USER_GUIDE.md-д бий.

CLI-ийн -b горим дундаж, медиан, хамгийн бага ба их хугацааг харуулна.
CLI-ийн харьцааг дунджаар, Dashboard-ийн харьцааг медианаар тооцдог.